

Университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина «*Тестирование программного обеспечения*»

Отчёт
По лабораторной работе №2

Вариант: 4734674

Студент:

Саранча П. А.

группа P3309

Преподаватель:

Наумова Н. А.

г. Санкт-Петербург, 2026 г.

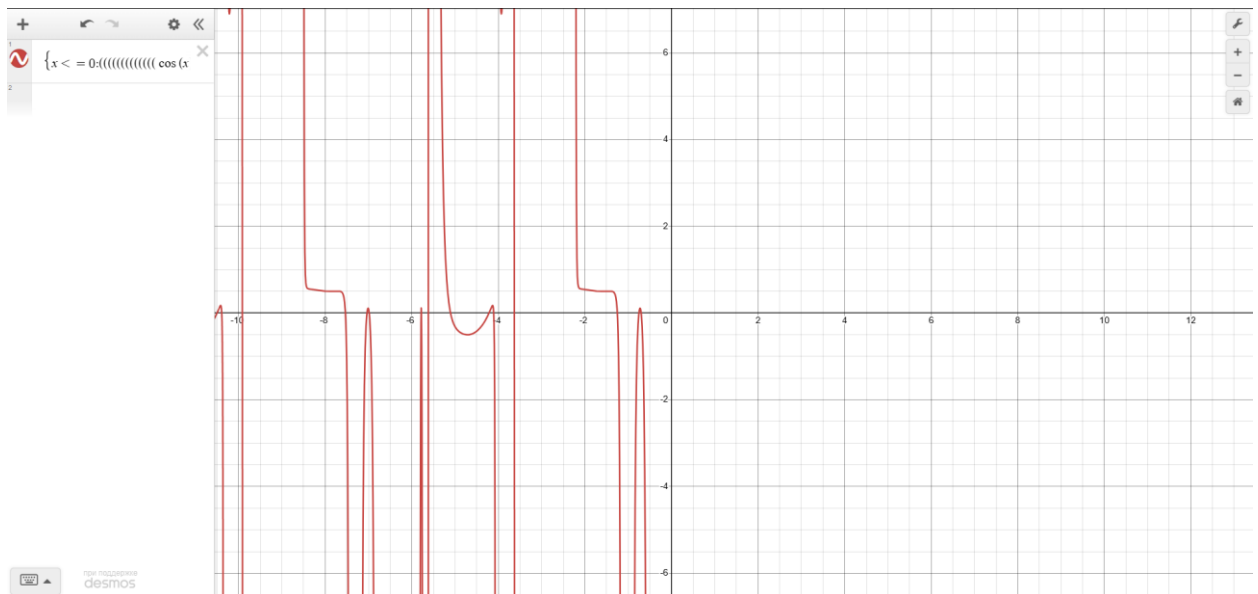
Описание задания

Введите вариант: 4734674

$$\left\{ \begin{aligned} & \left(\left(\left(\left(\left(\left(\left(\left(\frac{\left(\left(\left(\left(\left(\left(\left(\frac{\cos(x)}{\sin(x)} \right) \cdot \sin(x)^3 \right)^2 \right) - \cot(x) \right) - \sec(x) \right) - \cos(x) \right)^2 \right) \right) \right) \right) \cdot \left(\frac{\cot(x)}{\cot(x)} \right) \right) - \sin(x) \right) + \left(\left(\frac{\tan(x)}{\left(((\tan(x) - \csc(x)) + \cos(x))^2 \right)^3} \right) \cdot (\sec(x) - \cot(x)) \right) \right) \\ & - \left(\left(\left(\left(\left(\frac{\cos(x) \cdot ((\csc(x) - \sin(x))^2)}{\left(\frac{(\tan(x)^2) \cdot ((\cot(x) + \sin(x)) \cdot \cos(x))}{\cos(x)} \right)^3} \right) \cdot \left(\cot(x) \cdot \left(\left(\left(\frac{\sin(x) \cdot (\cos(x)^3)}{\sin(x)} \right)^2 \right) - \sin(x) \right) \right) \right) \right) \right) \right) \right) \text{ if } x \leq 0 \\ & \left(\left(\left(\left(\frac{\log_3(x) + \ln(x)}{\log_2(x)} \right) - (\ln(x) \cdot \log_5(x)) \right)^2 \right) - (\log_2(x) \cdot \log_{10}(x)) \right) \text{ if } x > 0 \end{aligned} \right.$$

$$\begin{aligned} x <= 0 : & (((((((((((((\cos(x) / \sin(x)) * \sin(x)) ^ 3) ^ 2) - \cot(x) - \sec(x)) * \cos(x)) ^ 2) / (\csc(x) \\ & + (\sin(x) ^ 3))) * (\cot(x) / \cot(x))) - \sin(x) + ((\tan(x) / (((\tan(x) - \csc(x)) + \cos(x)) ^ 2) ^ 3)) \\ & * (\sec(x) - \cot(x)))) - (((\cos(x) * ((\csc(x) - \sin(x)) ^ 3)) / (((\tan(x) ^ 3) * ((\cot(x) + \sin(x)) * \\ & \cos(x))) / \cos(x)) ^ 3)) * (\cot(x) * (((\sin(x) * (\cos(x) ^ 3)) / \sin(x)) ^ 2) - \sin(x)))) \end{aligned}$$

$$x > 0 : (((((\log_3(x) + \ln(x)) / \log_2(x)) - (\ln(x) * \log_5(x))) ^ 2) - (\log_2(x) * \log_{10}(x)))$$

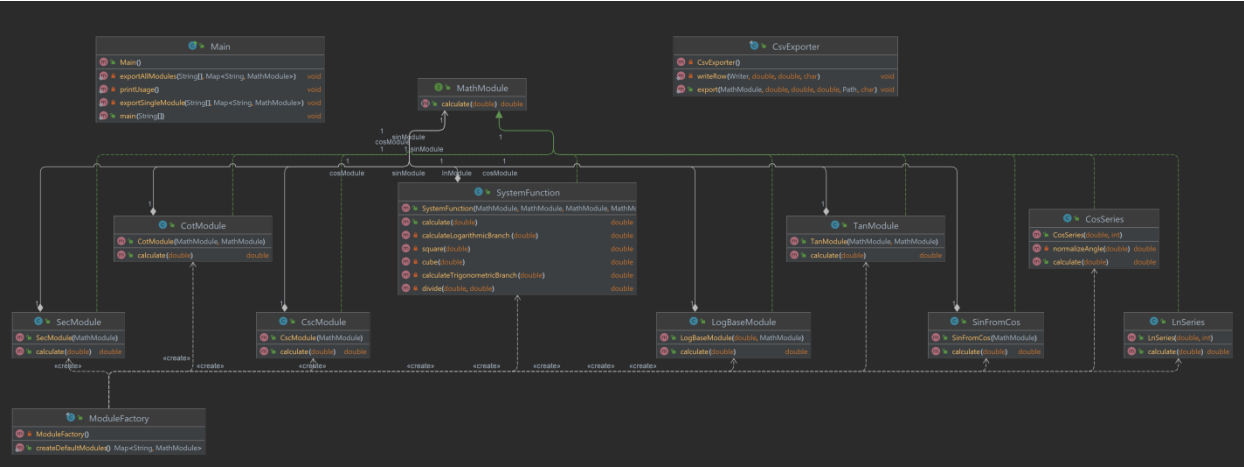


Выполнение

Исходный код

<https://github.com/PaulLocust/tpo-lab2>

UML



Описание тестового покрытия с обоснованием его выбора

Тестовое покрытие построено по принципу сочетания модульного и интеграционного тестирования с поэтапной заменой заглушек на реальные модули. Это позволило проверить как корректность отдельных вычислительных модулей, так и корректность их совместной работы в составе системы функций.

1. Покрытие модульных тестов базовых функций. Проверяются базовые функции, от которых зависят остальные модули: косинус через ряд и натуральный логарифм через ряд. Реализация: **CosSeriesTest.java**, **LnSeriesTest.java**. Обоснование: именно ошибки в базовых функциях дают систематические отклонения во всех производных модулях, поэтому их корректность проверяется отдельно и в первую очередь.
2. Покрытие зависимостей и производных функций. Проверено, что $\sin$ вычисляется через зависимость от $\cos$, а также корректность $\tan$, $\cot$, $\sec$, $\csc$ и логарифмов по основанию через $\ln$. Реализация: **SinFromCosTest.java**, **DerivedTrigModulesTest.java**, **LogBaseModuleTest.java**. Обоснование: это подтверждает соответствие варианту задания и корректность архитектурной зависимости модулей.
3. Анализ эквивалентных классов входных данных. Выделены и проверены классы: $x \leq 0$ для тригонометрической ветви и $x > 0$ для логарифмической ветви; дополнительно проверены валидные и невалидные области определения модулей. Реализация системного уровня: **SystemFunctionFullIntegrationTest.java**, **SystemFunctionLogIntegrationTest.java**, **SystemFunctionTrigIntegrationTest.java**. Обоснование: эквивалентные классы уменьшают число тестов без потери качества проверки поведения системы.
4. Проверка особенностей функции: асимптоты, окрестности выколотых точек, периодичность. Проверены точки и окрестности, где функция не определена или резко меняет поведение: нули знаменателей в тригонометрических выражениях, логарифмические ограничения, точки разрыва и большие значения около асимптот; отдельно проверена периодичность тригонометрических модулей. Обоснование: именно эти области создают наибольший риск дефектов и численной нестабильности.
5. Интеграционная стратегия по модулям с табличными заглушками. Интеграция выполнена поэтапно: сначала полностью на заглушках, затем по очереди подключаются реальные ветви (тригонометрическая и логарифмическая), после этого проводится полная интеграция. Реализация: **SystemFunctionWithTableStubsTest.java**, **SystemFunctionTrigIntegrationTest.java**,

SystemFunctionLogIntegrationTest.java,
SystemFunctionFullIntegrationTest.java. Обоснование: такой порядок упрощает локализацию ошибок и дает прозрачный контроль корректности на каждом шаге интеграции.

6. Использование параметризованных тестов и Mockito. Параметризованные тесты применены для покрытия множества реперных точек без дублирования кода; Mockito использован для табличных заглушек в интеграционных тестах. Обоснование: повышается воспроизводимость, читаемость и полнота проверки граничных сценариев.
7. Покрытие инфраструктуры экспорта данных для графиков. Проверены сценарии формирования CSV и запусков через точку входа приложения. Реализация: **CsvExporterTest.java**, **MainTest.java**. Обоснование: в лабораторной графике являются частью результата, поэтому корректность подготовки данных (CSV) также входит в область тестирования.

Тестовое покрытие

Test Summary

86
tests

0
failures

0
ignored

1.813s
duration

100%
successful

Packages

Classes

Package	Tests	Failures	Ignored	Duration	Success rate
lab.pavel	6	0	0	0.129s	100%
lab.pavel.function.log	22	0	0	1.495s	100%
lab.pavel.function.system	31	0	0	0.083s	100%
lab.pavel.function.trig	21	0	0	0.051s	100%
lab.pavel.io	6	0	0	0.055s	100%

Generated by [Gradle 8.5](#) at 19 anp. 2026 г., 20:05:42

Тесты успешно проходятся.

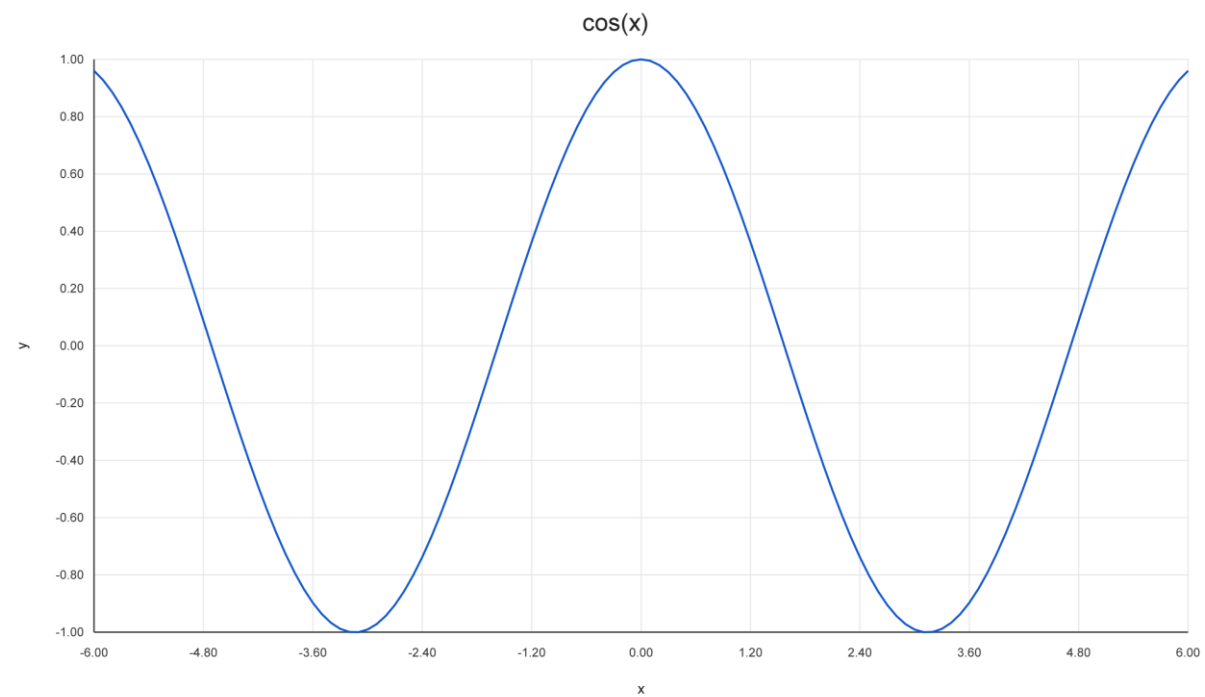
tpo-lab2

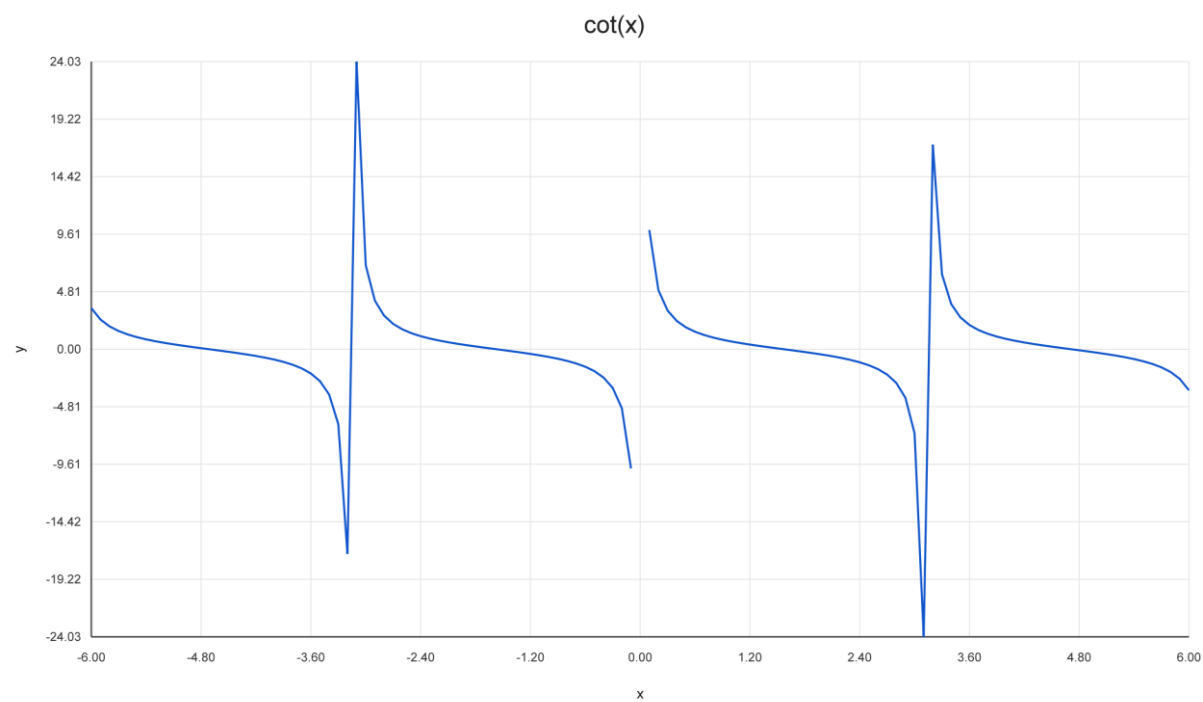
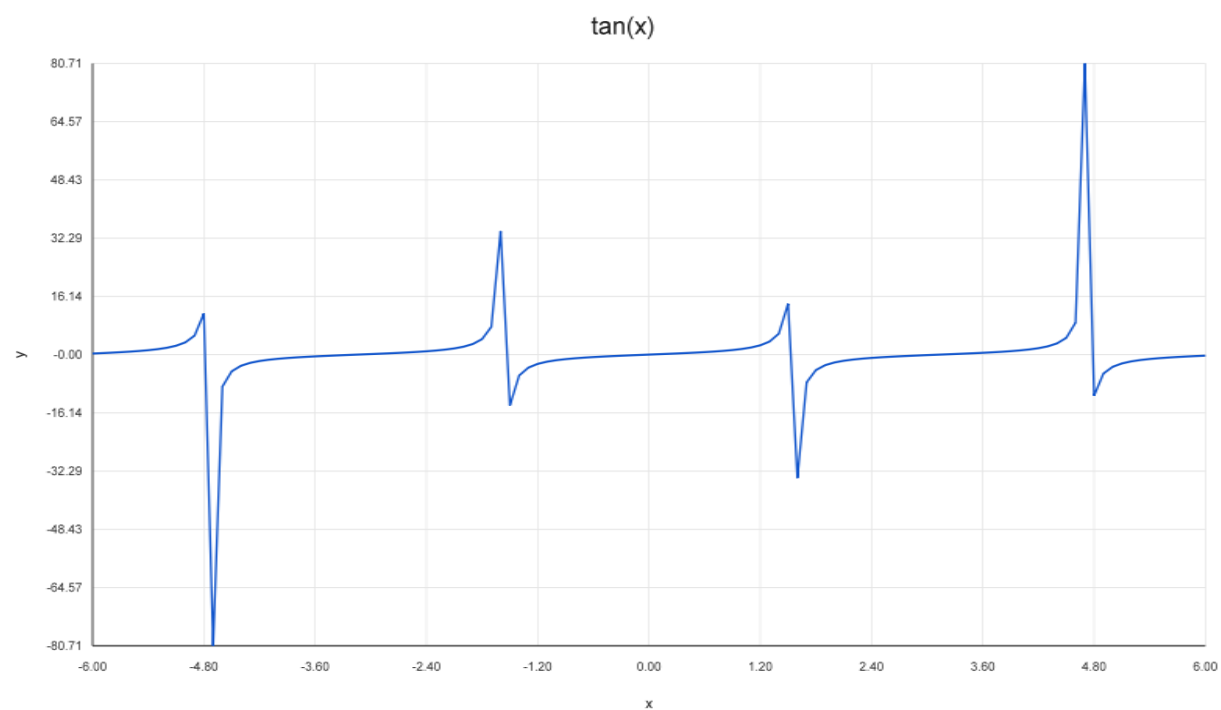
tpo-lab2

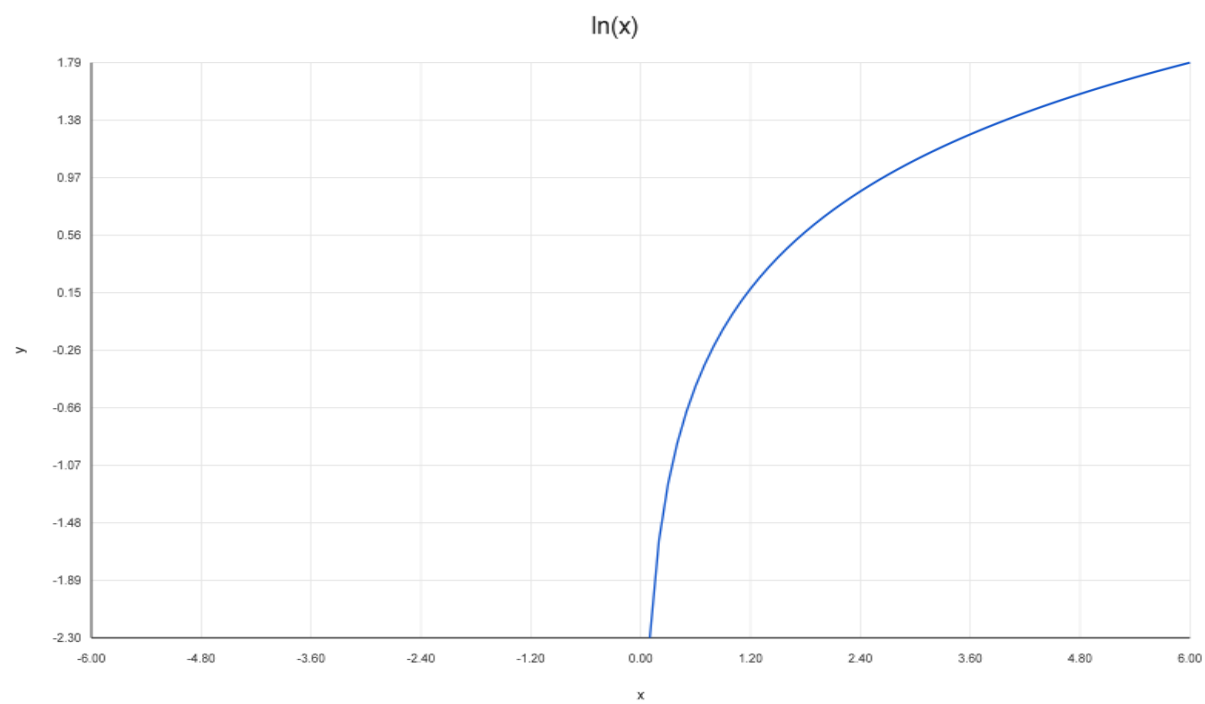
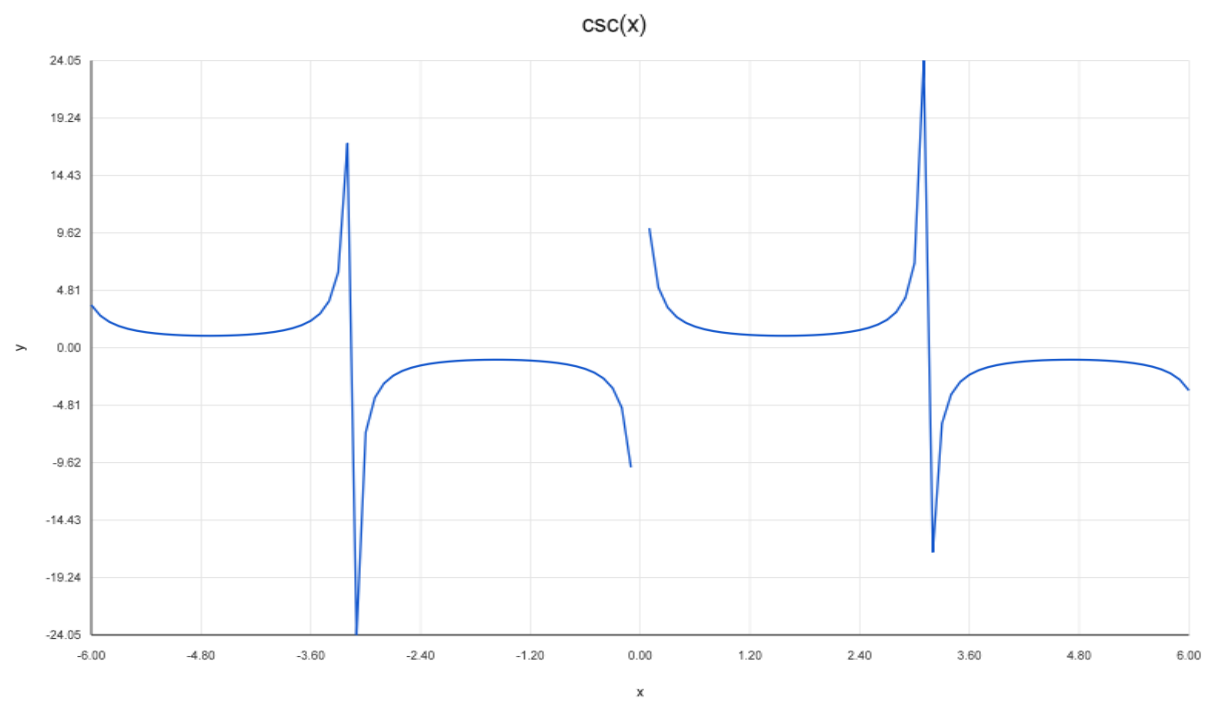
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
lab.pavel.function.system		100 %		100 %	0 11	0 52	0 7	0 1
lab.pavel		100 %		100 %	0 14	0 51	0 5	0 1
lab.pavel.function.trig		100 %		100 %	0 21	0 56	0 13	0 6
lab.pavel.function.factory		100 %		n/a	0 1	0 26	0 1	0 1
lab.pavel.function.log		100 %		100 %	0 12	0 30	0 4	0 2
lab.pavel.io		100 %		100 %	0 10	0 18	0 2	0 1
Total	0 of 1 051	100 %	0 of 74	100 %	0 69	0 233	0 32	0 12

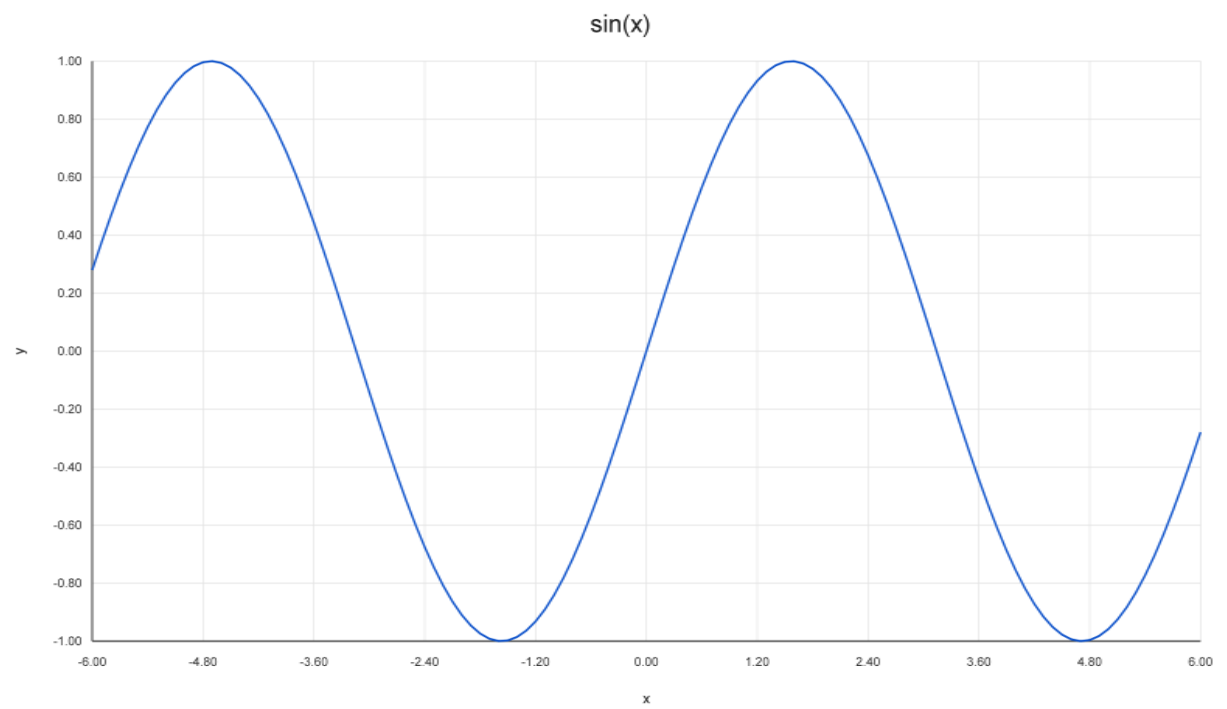
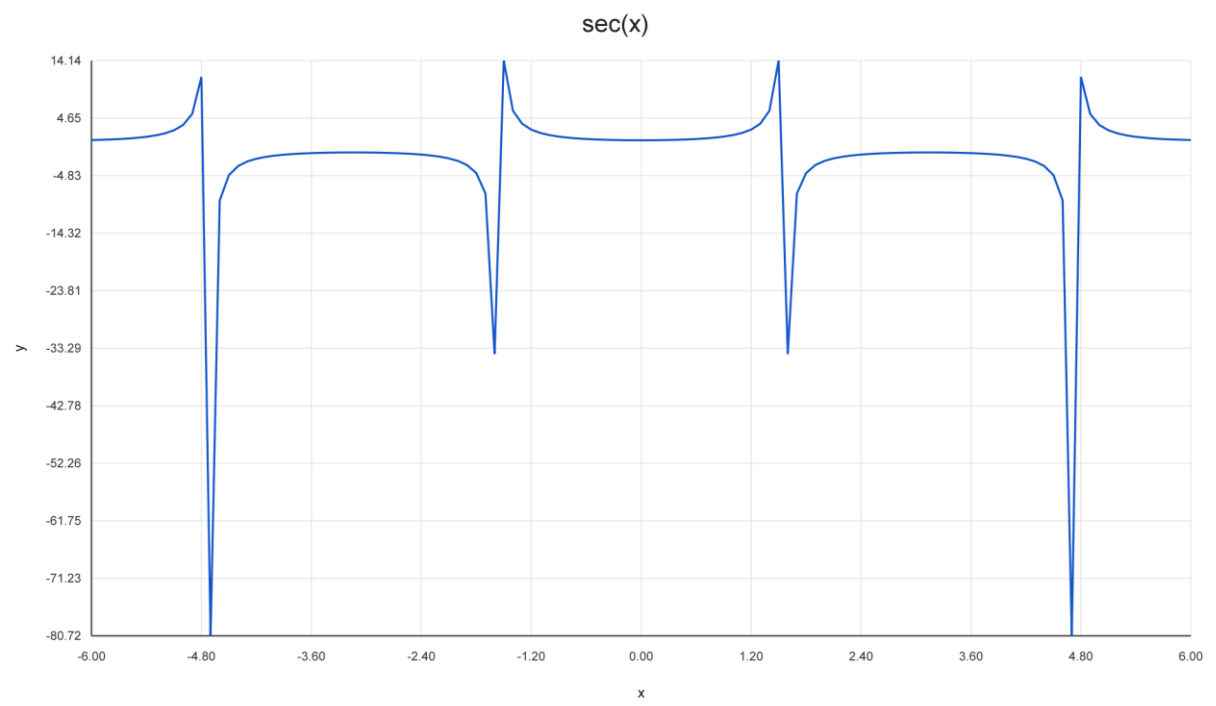
Итоговое тестовое покрытие.

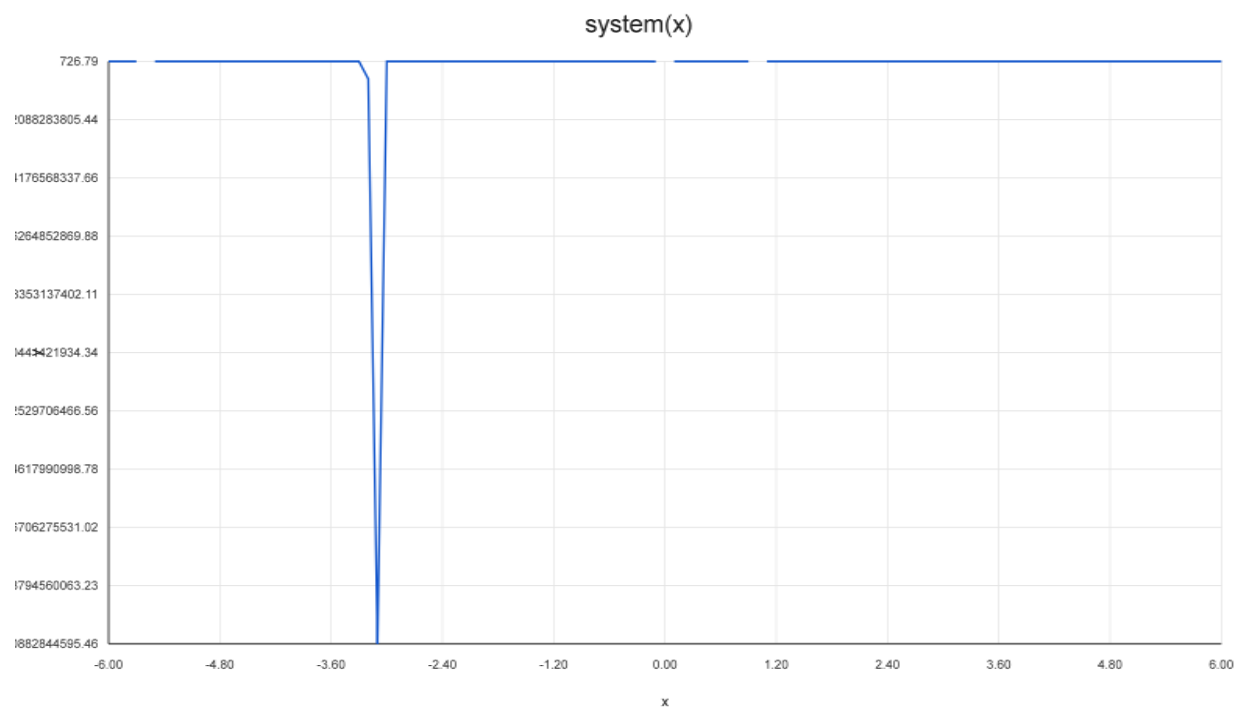
Графики











Выводы

В ходе выполнения лабораторной работы была реализована система функций, основанная на разложении в ряд. Проведено модульное и интеграционное тестирование с использованием JUnit 5 и Mockito, включая анализ граничных значений классов эквивалентности. Кроме того я научился тестировать приложения, работающие с дисплеем, получил базовые навыки CI. Выгрузка данных в CSV построенные графики подтвердили корректность вычислений. Работа позволила закрепить навыки интеграционного тестирования, работы с численными методами и проектирования модульных приложений.